

数据结构内部排序学习报告

成员分工

王枫：代码和学习报告编写

沈子力：演讲，书本文字提取

秦朗：

一、概念

排序的目标，是将一组记录按照关键字重新排列，使关键字满足递增或递减关系。若有记录序列 $R_1, R_2, \dots, R_n$ ，对应关键字为 $K_1, K_2, \dots, K_n$ ，排序的结果就是找到一种下标排列 $p_1, p_2, \dots, p_n$ ，使得

$$K_{p_1} \leq K_{p_2} \leq \dots \leq K_{p_n}$$

如果整个排序过程都在内存中完成，称为内部排序；如果数据量过大，排序过程需要借助外部存储设备，称为外部排序。本报告只分析内部排序中已经学习到的插入类排序和交换类排序。

内部排序算法通常围绕两种基本操作展开：一是比较关键字大小，二是移动记录位置。不同算法的差别，主要体现在如何减少比较次数、如何减少移动次数，以及是否需要额外辅助空间。

排序算法还要关注稳定性。若两个记录关键字相等，排序前先出现的记录在排序后仍然排在前面，则该排序方法是稳定的；否则是不稳定的。稳定性在多关键字排序中非常重要。例如先按学号排好，再按成绩排序，如果成绩相同仍要求学号小者在前，就应选择稳定排序。

教材中的代码以记录数组为基础，可以抽象为如下记录结构：

```
typedef int KeyType;

typedef struct {
    KeyType key;
    OtherType other_data;
} RecordType;
```

其中 `key` 是排序关键字，`other_data` 表示记录中除关键字外的其他数据。教材中的数组下标通常从 1 开始，并用 `r[0]` 作为监视哨或临时存储单元。

二、插入类排序

插入类排序的基本思想是：维护一个已经有序的子序列，每次从无序部分取出一个元素，将它插入到有序子序列中的正确位置。直接插入排序、折半插入排序和希尔排序都属于这一类。

1. 直接插入排序

直接插入排序将第 i 个记录插入到前面 $i - 1$ 个已经有序的记录中。具体做法是：先保存当前待插入记录，然后从后向前扫描有序区间；凡是关键字大于待插入记录的元素，都向后移动一个位置；当找到不大于待插入记录的位置时，将待插入记录放入空位。

教材中的直接插入排序代码如下：

```
void InsSort(RecordType r[], int n) {
    /* 对记录数组r做直接插入排序，n为数组中待排序记录的数目 */
    int i, j;

    for (i = 2; i <= n; i++) {
        r[0] = r[i];          /* 将待插入记录存放到监视哨r[0]中 */
        j = i - 1;

        while (r[0].key < r[j].key) { /* 寻找插入位置 */
            r[j + 1] = r[j];
            j = j - 1;
        }

        r[j + 1] = r[0];      /* 将待插入记录插入已排序的序列 */
    }
}
```

这段代码的关键是 `r[0]`。它有两个作用：第一，保存待插入记录，避免移动元素时丢失原值；第二，作为监视哨简化边界控制。由于教材数组从 1 开始，当前元素不断向前比较时，`r[0]` 可以防止查找位置时越界。

直接插入排序的最好情况是原序列已经有序，此时每趟只需比较一次，不需要移动大量记录，时间复杂度为 $O(n)$ 。最坏情况是原序列逆序，此时每个元素都要一路前移，时间复杂度为 $O(n^2)$ 。其空间复杂度为 $O(1)$ 。

直接插入排序是**稳定排序**。因为判断条件是 `r[0].key < r[j].key`，当两个关键字相等时，前面的元素不会被后移，后出现的相等元素也就不会插到先出现的相等元素前面。

个人实现中使用的是 0 下标数组和普通变量保存待插入值：

```
int v = A[i];
int j = i - 1;

while (j >= 0 && A[j] > v)
{
    A[j + 1] = A[j];
    j--;
}

A[j + 1] = v;
```

这个写法更符合 C/C++ 常见数组习惯，优点是直观；教材写法的优势是通过监视哨减少边界判断。两种写法的算法思想一致，时间复杂度和空间复杂度也相同。

2. 折半插入排序

折半插入排序是对直接插入排序中“寻找插入位置”这一环节的改进。直接插入排序用顺序查找确定插入位置，而折半插入排序利用前 $i - 1$ 个记录已经有序的特点，用折半查找确定插入位置。

教材中的折半插入排序代码如下：

```
void BinSort(RecordType r[], int n) {
    /* 对记录数组r进行折半插入排序，n为数组的长度 */
    int i, j, low, high, mid;
    RecordType x;

    for (i = 2; i <= n; ++i) {
        x = r[i];
        low = 1;
        high = i - 1;

        while (low <= high) {          /* 确定插入位置，减少比较次数 */
            mid = (low + high) / 2;

            if (x.key < r[mid].key)
                high = mid - 1;
            else
                low = mid + 1;
        }

        for (j = i - 1; j >= low; --j)
            r[j + 1] = r[j];          /* 记录依次向后移动，移动次数不变 */

        r[low] = x;                    /* 插入记录 */
    }
}
```

折半插入排序的比较次数少于直接插入排序。插入第 i 个元素时，用折半查找确定位置，比较次数约为 $\log_2 i$ 。但是，确定插入位置后，仍然需要移动元素，因此总时间复杂度仍为 $O(n^2)$ 。

这说明排序算法不能只看“比较次数”。在顺序存储结构中，插入一个元素往往伴随一批元素后移。如果移动代价没有降低，即使查找插入位置更快，整体数量级也可能不变。

折半插入排序是稳定排序。代码中当 `x.key` 与 `r[mid].key` 相等时执行 `low = mid + 1`，这会把新元素插入到已有相等元素之后，从而保持相等关键字的原有次序。

3. 希尔排序

希尔排序又称缩小增量排序，是对直接插入排序的进一步改进。直接插入排序在序列基本有序、规模较小时效率较高；希尔排序正是先通过较大间隔的分组插入，让整个序列快速接近有序，然后逐步缩小间隔，最后进行一次间隔为 1 的插入排序。

教材中的一趟希尔插入排序代码如下：

```
void ShellInsert(RecordType r[], int n, int delta) {
    /* 对记录数组r做一趟希尔插入排序，n为数组的长度，delta为增量 */
    int i, j;
```

```

for (i = 1 + delta; i <= n; i++) {
    /* 1 + delta为第一个子序列的第二个元素的下标 */
    if (r[i].key < r[i - delta].key) {
        r[0] = r[i];          /* 备份r[i], 不做监视哨 */

        for (j = i - delta; j > 0 && r[0].key < r[j].key; j -= delta) {
            r[j + delta] = r[j];
        }

        r[j + delta] = r[0];
    }
}
}

```

教材中的完整希尔排序代码如下：

```

void shellsort(RecordType r[], int n, int delta[], int m) {
    /* 对记录数组r做希尔排序, n为数组r的长度, delta为增量数组, m为delta[]的长度 */
    int i;

    for (i = 0; i <= m - 1; ++i) {
        ShellInsert(r, n, delta[i]);
    }
}

```

希尔排序的关键不在某一次插入，而在增量序列。每一趟排序只处理相距 `delta` 的元素。当 `delta` 较大时，元素可以跨越较长距离移动，能够较快减少序列中的逆序；当 `delta` 最终变为 1 时，序列已经基本有序，此时直接插入排序的效率较高。

个人实现中写出了一趟按增量插入的核心：

```

int v = A[i];
int j = i - g;

while (j >= 0 && A[j] > v)
{
    A[j + g] = A[j];
    j -= g; //直插同款
}

A[j + g] = v;

```

希尔排序的时间复杂度与增量序列有关，教材中给出的一般估计为 $O(n^{1.5})$ ，优于直接插入排序的 $O(n^2)$ 。它的空间复杂度为 $O(1)$ 。由于分组插入可能让相等关键字跨组移动，改变原有相对次序，因此希尔排序是不稳定排序。

三、交换类排序

交换类排序通过交换逆序元素来完成排序。冒泡排序和快速排序都属于交换类排序，但二者效率差别很大：冒泡排序每次只交换相邻元素，而快速排序通过一次划分可能消除多个逆序。

1. 冒泡排序

冒泡排序反复扫描待排序序列，比较相邻两个记录的关键字。如果相邻记录逆序，就交换它们的位置。每一趟扫描后，当前未排序部分的最大关键字记录会被放到末尾。

教材中的冒泡排序代码可以整理如下：

```
void BubbleSort(RecordType r[], int n) {
    /* 对记录数组r做冒泡排序，n为数组的长度 */
    int i, j;
    int change;
    RecordType x;

    change = TRUE;

    for (i = 1; i <= n - 1 && change; ++i) {
        change = FALSE;

        for (j = 1; j <= n - i; ++j) {
            if (r[j].key > r[j + 1].key) {
                x = r[j];
                r[j] = r[j + 1];
                r[j + 1] = x;
                change = TRUE;
            }
        }
    }
}
```

这段代码中的 `change` 是冒泡排序的重要优化。如果某一趟扫描没有发生任何交换，说明序列已经有序，可以提前结束。没有这个标记时，即使序列已经排好，算法仍会继续执行剩余趟数。

个人实现中的冒泡排序固定执行 $n - 1$ 趟：

```
for (i = 1; i < 5; i++)
{
    for (j = 0; j < 5 - i; j++)
    {
        if (a[j] < a[j + 1])
        {
            swap(a[j], a[j + 1]);
        }
    }
}
```

这段代码实现的是降序排序，因为判断条件是 `a[j] < a[j + 1]`。如果要升序排序，应改成 `a[j] > a[j + 1]`。与教材相比，它缺少 `change` 提前结束优化。可以改为：

```
for (int i = 1; i < n; i++)
{
    bool change = false;
```

```

for (int j = 0; j < n - i; j++)
{
    if (a[j] > a[j + 1])
    {
        swap(a[j], a[j + 1]);
        change = true;
    }
}

if (!change) break;
}

```

冒泡排序的最好情况是原序列已经有序，加入 `change` 后只需一趟扫描，时间复杂度为 $O(n)$ 。最坏情况是原序列逆序，需要进行约 $n(n-1)/2$ 次比较，时间复杂度为 $O(n^2)$ 。空间复杂度为 $O(1)$ 。

冒泡排序是稳定排序。因为它只在相邻元素逆序时交换，若两个关键字相等，则不会交换，因此相等元素的相对次序保持不变。

2. 快速排序

<https://xiaomaoyang.top/2026/06/03/quicksort/>

快速排序是对冒泡排序思想的改进。冒泡排序通过相邻交换逐步消除逆序，一次交换通常只能消除一个逆序；快速排序通过选取基准记录，将序列划分为左右两个子表，一次划分可能消除多个逆序，因此平均效率更高。

快速排序的基本思想是：选择一个基准记录 `x`，使基准左侧记录的关键字都小于它，右侧记录的关键字都大于或等于它；然后递归地对左右两个子表继续排序。

教材中的完整快速排序代码如下：

```

void QKSort(RecordType r[], int low, int high) {
    /* 对记录数组r[low..high]用快速排序算法进行排序 */
    int pos;

    if (low < high) {
        pos = QKPass(r, low, high);    /* 以基准记录为界划分两个子表 */
        QKSort(r, low, pos - 1);      /* 对左部子表快速排序 */
        QKSort(r, pos + 1, high);      /* 对右部子表快速排序 */
    }
}

```

教材中的一趟快速排序划分代码如下：

```

int QKPass(RecordType r[], int low, int high) {
    /*
     * 用基准记录对r[low]~r[high]部分进行一趟划分，
     * 使基准记录之前所有记录的关键字小于基准记录，
     * 基准记录之后所有记录的关键字大于或等于基准记录。
     */
    RecordType x;

    x = r[low];    /* 选择基准记录 */
}

```

```

while (low < high) {
    while (low < high && r[high].key >= x.key)
        high--;
        /* high从右到左找小于x.key的记录 */

    if (low < high) {
        r[low] = r[high];
        low++;
        /* 将小于x.key的记录送入空单元r[low] */
    }

    while (low < high && r[low].key < x.key)
        low++;
        /* low从左到右找大于或等于x.key的记录 */

    if (low < high) {
        r[high] = r[low];
        high--;
        /* 将大于或等于x.key的记录送入空单元r[high] */
    }
}

r[low] = x;
return low;
/* 将基准记录保存到low=high的位置 */
/* 返回基准记录的位置 */
}

```

教材代码采用的是“挖坑法”。先把 `r[low]` 保存到变量 `x` 中，使 `r[low]` 成为空单元；再从右侧找小于基准的记录放到左侧空位，从左侧找大于或等于基准的记录放到右侧空位；当 `low` 与 `high` 相遇时，将基准记录放回该位置。

个人实现中采用的是 **双指针**：

```

int x = q[l], i = l - 1, j = r + 1;

while (i < j)
{
    do i++; while (q[i] < x);
    do j--; while (q[j] > x);

    if (i < j) swap(q[i], q[j]);
}

quicksort(q, l, j);
quicksort(q, j + 1, r);

```

这个写法与教材代码的分区方式不同。教材通过不断填补空位完成划分；个人实现通过两个指针向中间扫描，遇到左右两侧不符合要求的元素就直接交换。这个写法的优点是代码短，但**边界细节更敏感**。

其中 `do...while` 是一个有价值的实现细节。它能保证指针至少移动一次，避免在大量元素等于基准值时，`i` 和 `j` 都停在原地导致死循环。递归区间写成 `[l, j]` 和 `[j + 1, r]`。

快速排序的最好情况是每次划分都接近均分，此时时间复杂度为 $O(n \log n)$ 。平均情况下快速排序也为 $O(n \log n)$ ，并且常数因子较小，实际效率很高。最坏情况是每次基准都选到最大值或最小值，划分极不平衡，时间复杂度退化为 $O(n^2)$ 。

快速排序的空间复杂度来自递归栈。平均情况下递归深度约为 $O(\log n)$ ，因此辅助空间复杂度为 $O(\log n)$ ；最坏情况下递归深度可能达到 $O(n)$ 。

快速排序通常是不稳定排序。因为划分过程中的交换可能让相等关键字跨越彼此，改变原有相对次序。例如相等关键字分布在基准两侧时，交换和递归划分都可能破坏稳定性。

四、范围内算法对比

算法	类别	核心思想	最好时间	平均时间	最坏时间	空间复杂度	稳定性
直接插入排序	插入类	将当前元素插入前面有序区间	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
折半插入排序	插入类	用折半查找确定插入位置	比较次数较少	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序	插入类	按增量分组插入，逐步缩小增量	与增量有关	常见估计 $O(n^{1.5})$	与增量有关	$O(1)$	不稳定
冒泡排序	交换类	反复交换相邻逆序元素	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
快速排序	交换类	基准划分后递归处理子表	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	平均 $O(\log n)$	不稳定

从表中可以看出，插入类排序的重点是“把元素放入已有序区间”。直接插入排序实现简单，在基本有序时效率很高；折半插入排序减少了比较次数，但没有减少移动次数；希尔排序通过增量分组使序列快速接近有序，因此通常比直接插入排序更快，但失去了稳定性。

交换类排序的重点是“通过交换消除逆序”。冒泡排序只交换相邻元素，理解简单但效率较低；快速排序通过一次划分消除大量逆序，平均性能明显更好，但需要注意基准选择和递归边界。

五、综合分析

本阶段学习的排序算法可以围绕两条主线理解。

第一条主线是插入思想。直接插入排序从“已有序区间”出发，逐个插入新元素；折半插入排序优化的是查找插入位置的过程；希尔排序优化的是序列整体状态，先通过大步长移动减少逆序，再利用直接插入排序在基本有序时效率高的特点完成最终排序。

第二条主线是交换思想。冒泡排序通过相邻交换逐步把最大值推到末尾，过程直观但效率有限；快速排序通过基准划分把序列分成两个子表，再递归处理，平均复杂度从 $O(n^2)$ 降到 $O(n \log n)$ 。

在实际选择算法时，需要结合数据规模、初始有序程度、稳定性要求和最坏情况风险。若数据规模较小或基本有序，直接插入排序简单有效；若希望在中等规模数据上获得比直接插入更好的效果，可以考虑希尔排序；若追求平均效率，快速排序更有优势；若必须保持稳定性，则直接插入排序、折半插入排序和冒泡排序更合适。